SOFTWARE ENGINEERING COURSE

LECTURE 10

Software Quality and Configuration Management

Dr. Tuape Micheal

MODULE 1 SCHEDULE

Week	Dates	Topic	SE Chapters
Week 9	Oct. 28–Nov1, 2024	Software Testing & Quality Management	Chapter 8 & 24
Week 10	Nov. 4– Nov 8, 2024	Software Evolution & Configuration Management	Chapter 9 & 25
Week 11	Nov. 11– Nov 15, 2024	Software Project Management & Software Project Planning	Chapter 22 & 23
Week 12	Nov. 18– Nov 22, 2024	Software Quality and Configuration	Chapter 24 & 25
Week 13	Nov. 25– Nov 29, 2024	Guest Lecture: Customer Success & Software Development	None
Week 14	December 2–6, 2023	Trends in Software Engineering	None

LEARNING OBJECTIVES

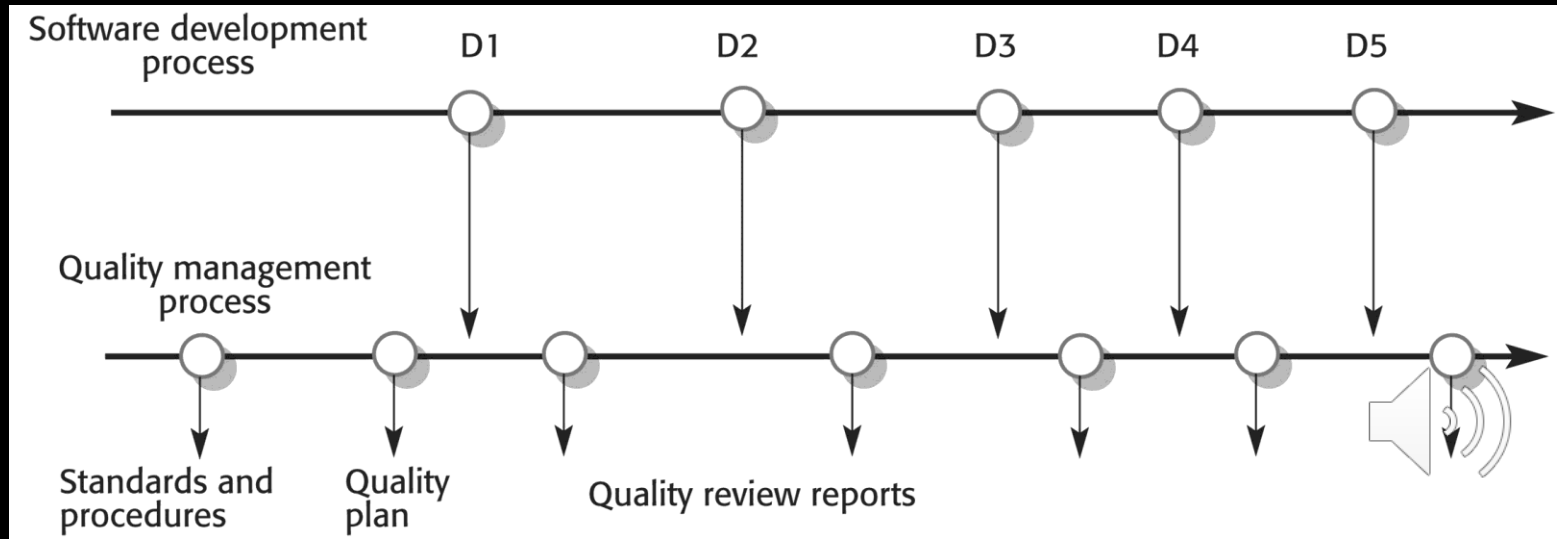
1. Software quality
2. Software standards
3. Quality management and agile development
4. Version management
5. System building
6. Change management
7. Release management

SOFTWARE QUALITY MANAGEMENT

- Concerned with ensuring that the required level of quality is achieved in a software product.
- Three principal concerns:
  - At the organizational level, quality management is concerned with establishing a framework of organizational processes and standards that will lead to high-quality software.
  - At the project level, quality management involves the application of specific quality processes and checking that these planned processes have been followed.
  - At the project level, quality management is also concerned with establishing a quality plan for a project. The quality plan should set out the quality goals for the project and define what processes and standards are to be used.

QUALITY MANAGEMENT ACTIVITIES

- Quality management provides an independent check on the development process.
- The quality management process checks the project deliverables to ensure that they are consistent with organizational standards and goals
- The quality team should be independent from the development team so that they can take an objective view of the software. This allows them to report on software quality without being influenced by software development issues.



QUALITY PLANNING

- A quality plan sets out the desired product qualities and how these are assessed and defines the most significant quality attributes.
- The quality plan should define the quality assessment process.
- It should set out which organisational standards should be applied and, where necessary, define new standards to be used.

QUALITY PLANS

- Quality plan structure
  - Product introduction;
  - Product plans;
  - Process descriptions;
  - Quality goals;
  - Risks and risk management.
- Quality plans should be short, succinct documents
  - If they are too long, no-one will read them.

SCOPE OF QUALITY MANAGEMENT

- Quality management is particularly important for large, complex systems. The quality documentation is a record of progress and supports continuity of development as the development team changes.
- For smaller systems, quality management needs less documentation and should focus on establishing a quality culture.
- Techniques have to evolve when agile development is used.

SOFTWARE QUALITY

- Quality, simplistically, means that a product should meet its specification.
- This is problematical for software systems
  - There is a tension between customer quality requirements (efficiency, reliability, etc.) and developer quality requirements (maintainability, reusability, etc.);
  - Some quality requirements are difficult to specify in an unambiguous way;
  - Software specifications are usually incomplete and often inconsistent.
- The focus may be ‘fitness for purpose’ rather than specification conformance.

SOFTWARE FITNESS FOR PURPOSE

- Has the software been properly tested?
- Is the software sufficiently dependable to be put into use?
- Is the performance of the software acceptable for normal use?
- Is the software usable?
- Is the software well-structured and understandable?
- Have programming and documentation standards been followed in the development process?

NON-FUNCTIONAL CHARACTERISTICS

- The subjective quality of a software system is largely based on its non-functional characteristics.
- This reflects practical user experience - if the software's functionality is not what is expected, then users will often just work around this and find other ways to do what they want to do.
- However, if the software is unreliable or too slow, then it is practically impossible for them to achieve their goals.

SOFTWARE QUALITY ATTRIBUTES

Safety	Understandability	Portability
Security	Testability	Usability
Reliability	Adaptability	Reusability
Resilience	Modularity	Efficiency
Robustness	Complexity	Learnability

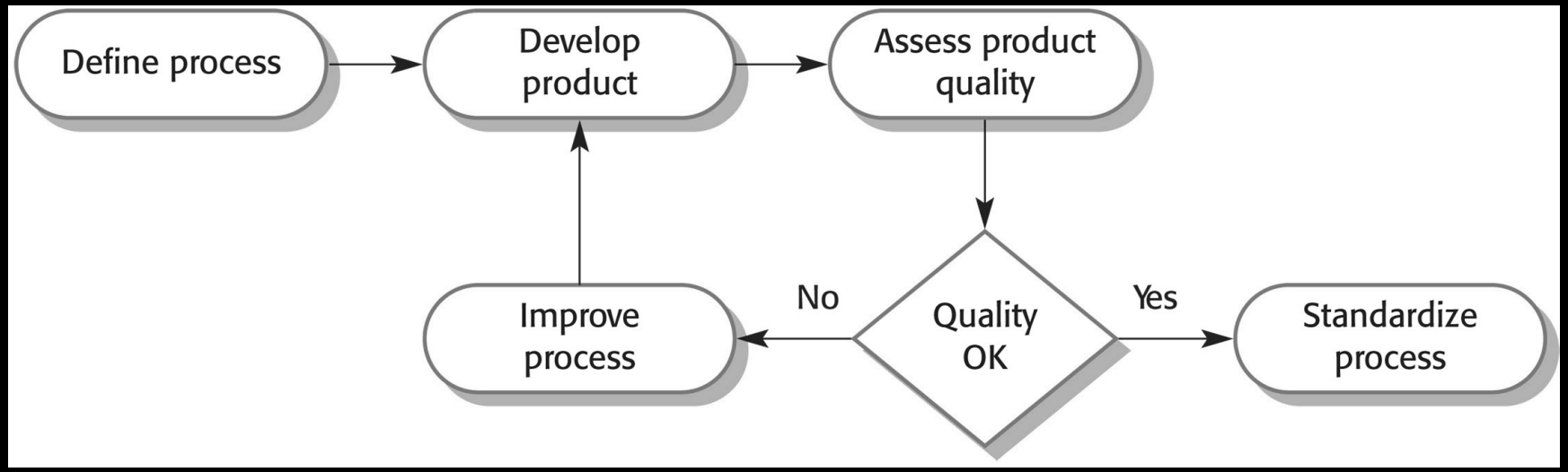
QUALITY CONFLICTS

- It is not possible for any system to be optimized for all of these attributes - for example, improving robustness may lead to loss of performance.
- The quality plan should therefore define the most important quality attributes for the software that is being developed.
- The plan should also include a definition of the quality assessment process, an agreed way of assessing whether some quality, such as maintainability or robustness, is present in the product.

PROCESS AND PRODUCT QUALITY

- The quality of a developed product is influenced by the quality of the production process.
- This is important in software development as some product quality attributes are hard to assess.
- However, there is a very complex and poorly understood relationship between software processes and product quality.
  - The application of individual skills and experience is particularly important in software development;
  - External factors such as the novelty of an application or the need for an accelerated development schedule may impair product quality.

PROCESS-BASED QUALITY



QUALITY CULTURE

- Quality managers should aim to develop a ‘quality culture’ where everyone responsible for software development is committed to achieving a high level of product quality.
- They should encourage teams to take responsibility for the quality of their work and to develop new approaches to quality improvement.
- They should support people who are interested in the intangible aspects of quality and encourage professional behavior in all team members.

SOFTWARE STANDARDS

- Standards define the required attributes of a product or process. They play an important role in quality management.
- Standards may be international, national, organizational or project standards.
- Encapsulation of best practice- avoids repetition of past mistakes.
- They are a framework for defining what quality means in a particular setting i.e. that organization’s view of quality.
- They provide continuity - new staff can understand the organisation by understanding the standards that are used.

PRODUCT AND PROCESS STANDARDS

- Product standards: Apply to the software product being developed.
- Process standards: These define the processes that should be followed during software development.

Product standards	Process standards
Design review form	Design review conduct
Requirements document structure	Submission of new code for system building
Method header format	Version release process
Java programming style	Project plan approval process
Project plan format	Change control process
Change request form	Test recording process

PROBLEMS WITH STANDARDS

- They may not be seen as relevant and up-to-date by software engineers.
- They often involve too much bureaucratic form filling.
- If they are unsupported by software tools, tedious form filling work is often involved to maintain the documentation associated with the standards.

QUALITY MANAGEMENT AND AGILE DEVELOPMENT

- Quality management in agile development is informal rather than document-based.
- It relies on establishing a quality culture, where all team members feel responsible for software quality and take actions to ensure that quality is maintained.
- The agile community is fundamentally opposed to what it sees as the bureaucratic overheads of standards-based approaches and quality processes.

SHARED GOOD PRACTICE

- Check before check-in
  - Programmers are responsible for organizing their own code reviews with other team members before the code is checked in to the build system.
- Never break the build
  - Team members should not check in code that causes the system to fail. Developers have to test their code changes against the whole system and be confident that these work as expected.
- Fix problems when you see them
  - If a programmer discovers problems or obscurities in code developed by someone else, they can fix these directly rather than referring them back to the original developer.

REVIEWS AND AGILE METHODS

- The review process in agile software development is usually informal.
- In Scrum, there is a review meeting after each iteration of the software has been completed (a sprint review), where quality issues and problems may be discussed.
- In Extreme Programming, pair programming ensures that code is constantly being examined and reviewed by another team member.

AGILE QM AND LARGE SYSTEMS

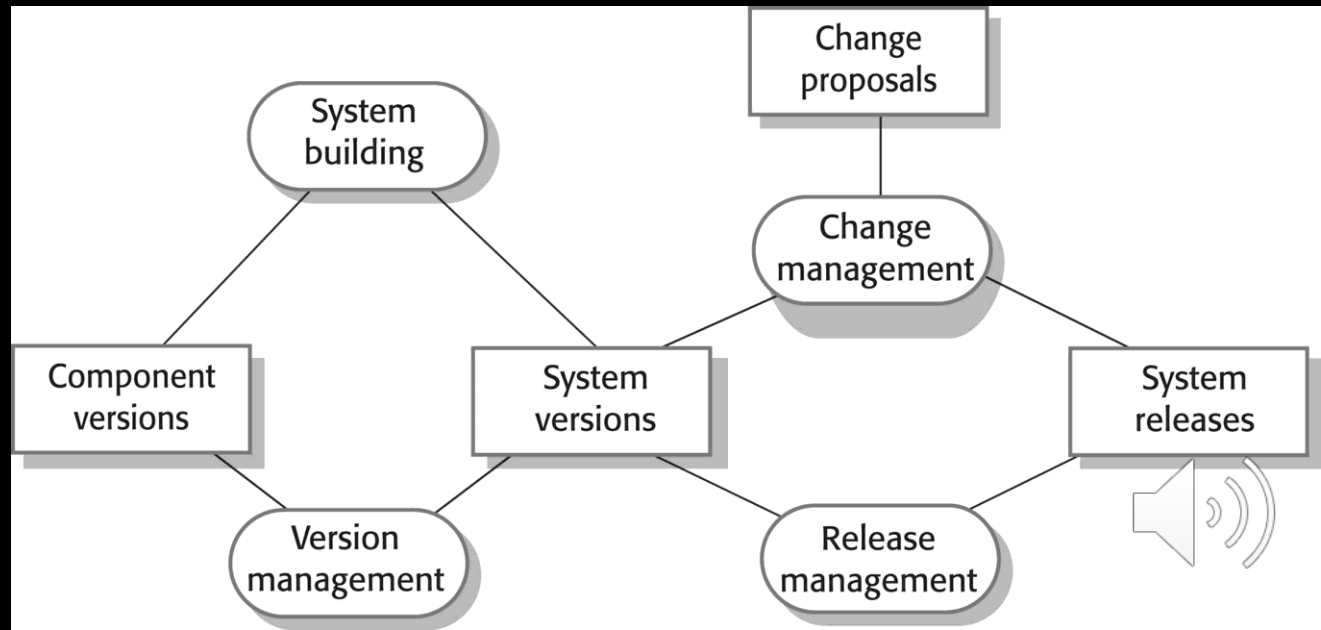
- When a large system is being developed for an external customer, agile approaches to quality management with minimal documentation may be impractical.
  - If the customer is a large company, it may have its own quality management processes and may expect the software development company to report on progress in a way that is compatible with them.
  - Where there are several geographically distributed teams involved in development, perhaps from different companies, then informal communications may be impractical.
  - For long-lifetime systems, the team involved in development will change without documentation, new team members may find it impossible to understand development.

CONFIGURATION MANAGEMENT

- Software systems are constantly changing during development and use.
- Configuration management (CM) is concerned with the policies, processes and tools for managing changing software systems.
- You need CM because it is easy to lose track of what changes and component versions have been incorporated into each system version.
- CM is essential for team projects to control changes made by different developers

CONFIGURATION MANAGEMENT ACTIVITIES

- **Version management:** Keeping track of the multiple versions of system components and ensuring that changes made to components by different developers do not interfere with each other.
- **System building:** The process of assembling program components, data and libraries, then compiling these to create an executable system.
- **Change management:** Keeping track of requests for changes to the software from customers and developers, working out the costs and impact of changes, and deciding the changes should be implemented.
- **Release management:** Preparing software for external release and keeping track of the system versions that have been released for customer use.



AGILE DEVELOPMENT AND CM

- Agile development, where components and systems are changed several times per day, is impossible without using CM tools.
- The definitive versions of components are held in a shared project repository and developers copy these into their own workspace.
- They make changes to the code then use system building tools to create a new system on their own computer for testing. Once they are happy with the changes made, they return the modified components to the project repository.


LUT
University